
Tarea 1

Una Neurona Artificial Construida Desde Cero

Curso: Machine Learning
Modalidad: Individual o parejas
Entregable: Notebook .ipynb + Informe .pdf

*“El mejor camino para entender una red neuronal es
construir su átomo fundamental: una sola neurona.”*

1. Contexto y Motivación

Una red neuronal profunda, sin importar su tamaño, está compuesta por unidades elementales llamadas **neuronas artificiales**. En esta tarea construirás una sola neurona —el bloque más básico del aprendizaje automático— y la entrenarás para predecir datos con estructura **lineal** y **cuadrática**, ambos contaminados con ruido gaussiano.

Al hacerlo sin recurrir a ninguna librería de machine learning, pondrás en práctica directamente los principios matemáticos del gradiente descendente, la función de pérdida y la propagación hacia atrás.

Objetivo de Aprendizaje

Al finalizar esta tarea serás capaz de:

- Implementar desde cero la computación $z = \mathbf{w}^\top \mathbf{x} + b$ y su gradiente.
- Aplicar la técnica de **linealización de características** para ajustar relaciones no lineales con una neurona lineal.
- Visualizar interactivamente cómo los hiperparámetros afectan el entrenamiento.
- Comunicar resultados técnicos mediante gráficas bien elaboradas y texto explicativo.

Restricción importante

Queda **prohibido** usar cualquier librería de machine learning: `scikit-learn`, `TensorFlow`, `PyTorch`, `Keras`, `JAX`, etc. Solo están permitidas `numpy` (para álgebra), `matplotlib` / `plotly` (para gráficas) e `ipywidgets` (para interactividad). El uso de librerías prohibidas anulará la nota de la parte correspondiente.

2. Fundamento Matemático

2.1. La neurona lineal

Dado un vector de entrada $\mathbf{x} \in \mathbb{R}^d$, la neurona calcula:

$$\hat{y} = z = \mathbf{w}^\top \mathbf{x} + b = \sum_{j=1}^d w_j x_j + b \quad (1)$$

donde $\mathbf{w} \in \mathbb{R}^d$ es el vector de pesos y $b \in \mathbb{R}$ es el sesgo (*bias*). En el caso escalar ($d = 1$) esto se reduce a:

$$\hat{y} = wx + b \quad (2)$$

2.2. Función de pérdida

Usaremos el **Error Cuadrático Medio** (MSE) sobre un conjunto de N ejemplos $\{(x^{(i)}, y^{(i)})\}_{i=1}^N$:

$$\mathcal{L}(w, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2 \quad (3)$$

2.3. Gradiente Descendente (Batch)

Actualizamos los parámetros en dirección contraria al gradiente de $\mathcal{L}$:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{2}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)}) x^{(i)} \quad (4)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{2}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)}) \quad (5)$$

$$w \leftarrow w - \alpha \frac{\partial \mathcal{L}}{\partial w} \quad (6)$$

$$b \leftarrow b - \alpha \frac{\partial \mathcal{L}}{\partial b} \quad (7)$$

donde $\alpha > 0$ es la **tasa de aprendizaje** (*learning rate*).

2.4. Linealización de características para datos cuadráticos

Una neurona lineal *no puede* ajustar directamente $y = ax^2 + bx + c$. Sin embargo, si transformamos la entrada creando una nueva característica $x_2 = x^2$, el problema se convierte en uno **lineal en las características**:

$$\hat{y} = w_1 x + w_2 x^2 + b \quad (8)$$

Esta técnica se llama **expansión polinomial de características** y es un ejemplo fundamental de ingeniería de características. Tu neurona seguirá siendo la misma ecuación (1) con $d = 2$ y $\mathbf{x} = [x, x^2]^\top$.

3. Descripción de la Tarea

3.1. Parte 1 — Generación de Datos con Ruido (10 pts)

Genera dos conjuntos de datos sintéticos:

A. Datos Lineales: muestrea $N = 200$ puntos de

$$y = m x + c + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2) \quad (9)$$

con $x \in [-5, 5]$, y parámetros verdaderos m y c de tu elección.

B. Datos Cuadráticos: muestrea $N = 200$ puntos de

$$y = a x^2 + b x + c + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2) \quad (10)$$

con $x \in [-5, 5]$.

Requisitos de esta parte:

- Fija una semilla aleatoria (`np.random.seed`) para reproducibilidad.
- Divide cada conjunto en 80 % entrenamiento y 20 % prueba.
- Grafica los datos con sus respectivos conjuntos de entrenamiento y prueba, incluyendo la curva verdadera (sin ruido) como referencia.
- Experimenta con al menos **dos niveles de ruido** (σ bajo y alto) y discute cómo afectan visualmente a los datos.

3.2. Parte 2 — Implementación de la Neurona

Trata de que la neurona sea una función reutilizable paramétrica, tu decides si usar constructores y POO o solo con funciones y llamadas a estas, la arquitectura es libre.

3.3. Parte 3 — Entrenamiento y Resultados

3.1. Caso Lineal: entrena tu neurona directamente con x como única característica.

3.2. Caso Cuadrático: aplica la linealización descrita en la sección 2.4 y entrena con $\mathbf{x} = [x, x^2]^\top$.

Para cada caso, genera las siguientes gráficas:

- **Gráfica 1 — Curva de pérdida:** evolución del MSE de entrenamiento (y validación si aplica) a lo largo de las épocas. Usa escala logarítmica en el eje y si la convergencia es lenta.
- **Gráfica 2 — Ajuste del modelo:** dispersión de los datos de entrenamiento (puntos), datos de prueba (puntos con distinto color/marcador) y la curva predicha por el modelo final.
- **Gráfica 3 — Superficie de pérdida:** para el caso lineal, grafica $\mathcal{L}(w, b)$ como un mapa de calor o superficie 3D, marcando la trayectoria del gradiente descendente sobre ella.

Reporta w, b aprendidos vs. los valores verdaderos, y calcula MSE en el conjunto de prueba.

3.4. Parte 4 — Notebook Interactivo con Slides

Convierte tu notebook en una **presentación interactiva** usando `ipywidgets` y el modo *RISE* (o `nbconvert -to slides`).

Widgets obligatorios

Implementa al menos los siguientes controles interactivos:

Widget	Tipo	Efecto a visualizar
Tasa de aprendizaje α	Slider	Cambio en velocidad y estabilidad de convergencia
Número de épocas	Slider	Punto de parada del entrenamiento
Nivel de ruido σ	Slider	Efecto sobre el ajuste y la pérdida final
Inicialización de w_0	Slider	Distintos puntos de inicio en la superficie de pérdida
Grado del polinomio	Selector (1–4)	Subajuste vs. sobreajuste

Cuadro 1: Widgets mínimos requeridos en el notebook interactivo.

Estructura de las slides

Organiza el notebook en al menos 5 secciones-diapositiva:

1. Portada: título, nombres, fecha.
2. Generación de datos y exploración.
3. Implementación y verificación de la neurona.
4. Resultados: gráficas de ajuste y convergencia.
5. Exploración interactiva de hiperparámetros.

3.5. Parte 5 — Análisis y Discusión

Trata de saber responder:

- Q1.** ¿Qué ocurre con la convergencia cuando la tasa de aprendizaje es demasiado alta? ¿Cuándo es demasiado baja? Ilustra con gráficas de pérdida.
- Q2.** ¿Por qué la neurona lineal no puede ajustar datos cuadráticos sin la transformación de características? Demuéstralo experimentalmente.
- Q3.** ¿Qué efecto tiene inicializar todos los pesos en cero frente a inicialización aleatoria en este modelo de una sola neurona?
- Q4.** Si aumentas el grado del polinomio (p. ej. incluir x^3 , x^4) en los datos lineales, ¿qué problema potencial surge? ¿Qué observas en las gráficas?

- Q5.** Reflexión: ¿qué limitaciones fundamentales tiene una *única* neurona lineal que motivan la existencia de redes neuronales profundas?

*¡Mucho éxito! Recuerda que el error es parte del proceso de aprendizaje,
tanto para los modelos como para nosotros.*
